

Trabalho Prático de Avaliação #1 - Programação em Shell

(25% da nota final)

Tema: handin_mgr - Gestão de entregas e lista de tarefas

1. Objetivo

Desenvolver um programa em Shell (POSIX sh) chamado handin_mgr que suporte dois conjuntos de funcionalidades:

- A) Gestão de tarefas (to-do) persistente
- B) Gestão e validacao de entregas de trabalhos (hand-ins)

O programa deve funcionar em ambiente Unix/Linux com /bin/sh e apenas com ferramentas standard de sistema (find, grep, awk, sed, sort, uniq, cut, tr, wc, date, du, cp, mv, rm, mkdir, test). É permitido usar gzip, tar e unzip apenas se existirem no sistema (o programa deve detetar a sua existência e adaptar o comportamento).

2. Regras gerais

2.1. Interpretador e portabilidade

- O script deve iniciar com: #!/bin/sh
- Não é permitido usar bashismos (arrays, [[...]], process substitution, etc).
- O script deve suportar nomes de ficheiros e diretorias com espaços.

2.2. Interface

- Síntaxe geral: handin_mgr <comando> [opcoes] [args]
- Deve existir ajuda:
 - handin_mgr -h mostra ajuda geral e exemplos
 - handin_mgr <comando> -h mostra ajuda do comando

2.3. Robustez e erros

- Todos os erros devem ser escritos em stderr.
- O script deve devolver código de saída diferente de 0 em caso de erro.
- Argumentos inválidos ou combinações inválidas devem ser detetados.

2.4. Estado persistente

O programa deve criar e manter uma pasta de estado em:

- \$HOME/.handin_mgr/

Esta pasta deve conter, no mínimo:



- config (opcional, mas recomendado)
- todos.csv
- runs/ (relatórios de execução)
- locks/ (mecanismo de exclusão mútua)

2.5. Exclusão mútua

Para evitar corrupção do estado, o programa deve usar um lock quando escreve em ficheiros de estado (ex: todos.csv e runs/*).

Sugestao: lock baseado em mkdir atomic (ex: mkdir \$STATE/locks/main.lock).

3. Formatos e convencoes

3.1. Separador de campos

Todos os ficheiros CSV produzidos pelo programa devem usar ; como separador.

3.2. Timestamps

Sempre que se registre data/hora, deve ser em formato ISO:

- AAAA-MM-DD HH:MM:SS
Quando for necessario um id de execucao, deve ser usado um identificador baseado em timestamp:
- AAAA-MM-DD_HHMMSS

4. Parte A - Gestão de tarefas

As tarefas devem ser guardadas em \$HOME/.handin_mgr/todos.csv com o seguinte formato por linha:

id;status;prio;due;tags;created;done;title

Onde:

- id: inteiro positivo incremental
- status: OPEN ou DONE
- prio: inteiro (1 a 5), em que 5 e mais urgente
- due: AAAA-MM-DD ou vazio
- tags: lista separada por virgulas (ex: so,tp1,handin)
- created: timestamp ISO
- done: timestamp ISO ou vazio
- title: texto livre (pode conter espaços)

4.1. Comando todo-add

Sintaxe:

```
handin_mgr todo-add "titulo" [-p P] [-d AAAA-MM-DD] [-t tag1,tag2]
```

Requisitos:

- Cria uma nova tarefa com status OPEN.
- Se -p não for indicado, prio default é 3.
- Se -d não for indicado, due fica vazio.
- Se -t não for indicado, tags fica vazio.
- id deve ser 1 + maior id existente (ou 1 se o ficheiro não existir).
- O comando deve criar o ficheiro todos.csv se não existir.

Erros:

- Falta do titulo.
- Prioridade fora de 1..5.
- Data com formato inválido.

4.2. Comando todo-list

Sintaxe:

```
handin_mgr todo-list [-a] [-s sort] [-t tag] [-p minPrio]
```

Requisitos:

- Por omissão lista apenas tarefas OPEN.
- Com -a lista OPEN e DONE.
- Filtragem:
 - -t tag lista apenas tarefas que contenham essa tag.
 - -p minPrio lista apenas tarefas com prio $\geq$ minPrio.
- Ordenação:
 - -s due: ordenar por due crescente (tarefas sem due no fim)
 - -s prio: ordenar por prio decrescente
 - -s created: ordenar por created crescente
- Saída em formato legível, por linha, exemplo:

```
[12] (P4) 2026-03-01 OPEN tags:so,tp1 - Corrigir README
```

Erros:

- sort inválido (não seja due, prio ou created)
- minPrio fora de 1..5

4.3. Comando todo-done

Sintaxe:

handin_mgr todo-done <id>

Requisitos:

- Marca a tarefa com esse id como DONE.
- Preenche o campo done com timestamp atual.
- Se já estiver DONE, deve manter DONE e informar o utilizador (sem alterar done).

Erros:

- id inexistente
- id inválido

4.4. Comando todo-search

Sintaxe:

handin_mgr todo-search <texto>

Requisitos:

- Procura case-insensitive no campo title.
- Lista as tarefas correspondentes no mesmo formato do todo-list.

Erros:

- falta do texto

5. Parte B - Gestão de entregas (handin)

5.1. Convenção de nomes

Uma entrega e uma pasta ou um ficheiro comprimido com nome:

ALUNO_UC_TP#_YYYYMMDDHHMM

Exemplos:

a12345_SO_TP1_202602131845

b54321_SO_TP2_202602101210

Onde:

- ALUNO: identificador alfanumérico sem underscores
- UC: identificador alfanumérico sem underscores
- TP#: TP seguido de número (ex: TP1, TP2)

- timestamp: 12 dígitos AAAAMMDDHHMM

5.2. Estrutura exigida dentro da entrega

Ao considerar a raiz da entrega (pasta extraída ou pasta original), devem existir:

- README.md ou README.txt (obrigatório)
- pasta src/ (obrigatória) com pelo menos 1 ficheiro regular
- Makefile (opcional)

5.3. Conteúdos proibidos

Devem ser detetados e reportados como falha:

- Diretorias: node_modules, dist, build (em qualquer nível)
- Ficheiros binários (heurística: `grep -Iq . ficheiro falha`)
- Ficheiros com tamanho superior a LIMITE_MB (default 5MB)

5.4. Repositório de entregas

O repositório deve ser organizado assim:

<repo_dir>/<UC>/<TP#>/<ALUNO>/<timestamp>/

Exemplo:

repo/SO/TP1/a12345/202602131845/

6. Comandos handin

6.1. Comando handin-ingest

Sintaxe:

handin_mgr handin-ingest <inbox_dir> <repo_dir> [-m]

Função:

- Varre inbox_dir a procura de:
 - diretorias cujo nome respeite a convenção
 - ficheiros .zip, .tar, .tar.gz cujo nome base respeite a convenção
- Para cada entrega válida o nome e copia para o repositório.
- Se -m estiver presente, move em vez de copiar.
- Se a entrega for comprimida e existirem ferramentas para extrair:
 - deve extrair para o destino final
 - o conteúdo extraído e que será validado em handin-check



Relatório:

- Deve criar runs/ingest_<timestamp>.txt com linhas do tipo:

OK;origem;destino

FAIL;origem;motivo

Duplicados:

- Se existirem várias entregas do mesmo ALUNO+UC+TP#, devem ser mantidas.
- O relatório deve marcar as entregas antigas como SUPERSEDED quando existir uma mais recente.

Erros:

- inbox_dir inexistente ou sem permissões
- repo_dir inexistente ou sem permissões

6.2. Comando handin-check

Sintaxe:

handin_mgr handin-check <repo_dir> [-o relatorio]

Função:

- Percorre todas as entregas dentro de repo_dir.
- Para cada entrega:
 - valida nome e estrutura
 - confirma README e src/ com pelo menos 1 ficheiro
 - deteta proibidos (pastas, binarios, >5MB)
 - calcula métricas:
 - num_src_files: numero de ficheiros regulares em src/
 - total_files: numero total de ficheiros regulares
 - total_lines: total de linhas somadas de todos os ficheiros de texto em src/
- Produz um relatório:
 - se -o for indicado, escreve no ficheiro
 - senao escreve em stdout
- Formato por entrega (uma linha):
UC;TP#;ALUNO;timestamp;STATUS;num_src_files;total_files;total_lines;problemas

Onde:

- STATUS: OK ou FAIL
- problemas: vazio ou lista separada por virgulas (ex: missing_readme,binary_found,too_large)

Obrigatório: Integração com todo

- Sempre que uma entrega tiver STATUS FAIL, o script deve criar automaticamente uma tarefa OPEN em todos.csv com:
 - title: "Corrigir entrega ALUNO UC TP# (motivo)"
 - tags: handin,UC,TP#
 - prio: 5 se o erro for missing_src ou binary_found, senao 4

6.3. Comando handin-summary

Sintaxe:

handin_mgr handin-summary <repo_dir> [-u UC] [-t TP#]

Função:

- Gera um resumo agregado das entregas (filtrado por UC e/ou TP se indicado):
 - total de entregas
 - numero de OK e FAIL
 - top 5 entregas com mais ficheiros (total_files)
 - top 5 entregas com mais linhas (total_lines)
 - lista de alunos com pelo menos uma entrega FAIL (únicos)

Formato:

- Texto legível com secções e contagens.

7. Requisitos de implementacao

- Deve usar quoting correto em todas as expansões de variáveis e paths.
- Deve evitar ler ficheiros linha a linha sem necessidade quando uma pipeline resolve.
- Deve lidar com ficheiros inexistentes (ex: todos.csv) criando-os quando apropriado.
- Deve garantir que a criação de tarefas automáticas nao cria duplicados idênticos em repetidas execuções do handin-check (solução a escolha do grupo, mas deve estar documentada).

8. Entrega

- 1 ficheiro executável: handin_mgr
- 1 ficheiro README.txt (ou README.md) com:
 - descrição
 - como executar cada comando
 - exemplos
 - limitacoes e suposicoes
 - ferramentas opcionais usadas (tar/unzip/gzip) e como o script se comporta sem elas

9. Avaliação

- 35% Parte A (todo): add, list, done, search
- 45% Parte B (handin): ingest, check, summary



- 10% Integração (criar tarefas a partir de falhas)
- 10% Robustez e qualidade (locks, erros, portabilidade, nomes com espaços, ajuda)

10. Exemplos de utilizacao

1. Criar tarefa:
`handin_mgr todo-add "Rever enunciado TP1" -p 4 -d 2026-02-20 -t so,tp1`
2. Listar tarefas abertas ordenadas por prazo:
`handin_mgr todo-list -s due`
3. Marcar tarefa concluída:
`handin_mgr todo-done 3`
4. Ingerir entregas:
`handin_mgr handin-ingest /home/prof/inbox /home/prof/repo -m`
5. Validar e gerar relatório:
`handin_mgr handin-check /home/prof/repo -o /home/prof/check_tp1.csv`
6. Resumo:
`handin_mgr handin-summary /home/prof/repo -u SO -t TP1`

Erro ~~todo~~-search ✓